

```

program NewtonInterpolation
  implicit none
  integer, parameter :: n = 5
  real, dimension(5) :: x, y
  real :: x_interp, interpolatedValue
  integer :: i

  ! Data points initialization
  x = [1.0, 2.0, 4.0, 5.0, 7.0]
  y = [52.0, 5.0, -5.0, -40.0, 10.0]

  x_interp = 3.0

  ! Call the computed interpolated value
  call NewtonDividedDifference(n, x, y, x_interp, interpolatedValue)

  ! Output the interpolated value
  write(*,*) "The interpolated value at x =", x_interp, "is", interpolatedValue

end program NewtonInterpolation

subroutine NewtonDividedDifference(n, x, y, x_interp, phi)
  implicit none
  integer, intent(in) :: n
  real, intent(in) :: x(n), x_interp
  real, intent(inout) :: y(n)
  real, intent(out) :: phi
  integer :: i, k

  ! Compute divided differences
  do k = 2, n
    do i = n, k, -1
      y(i) = (y(i) - y(i-1)) / (x(i) - x(i-k+1))
    end do
  end do

  ! Compute interpolated value
  phi = y(n)
  do k = n-1, 1, -1
    phi = y(k) + (x_interp - x(k)) * phi
  end do
end subroutine NewtonDividedDifference

```